

LAB 6: SNN FOR CLASSIFICATION

Instructor

Michele Rossi - michele.rossi@unipd.it

Lab. classes

Eleonora Cicciarella - eleonora.cicciarella@phd.unipd.it

Cesare Bidini - cesare.bidini@phd.unipd.it



Lab 6

- SNN-based classifier

- PyTorch/snnTorch
- Image classification with static MNIST dataset

- You will learn to:

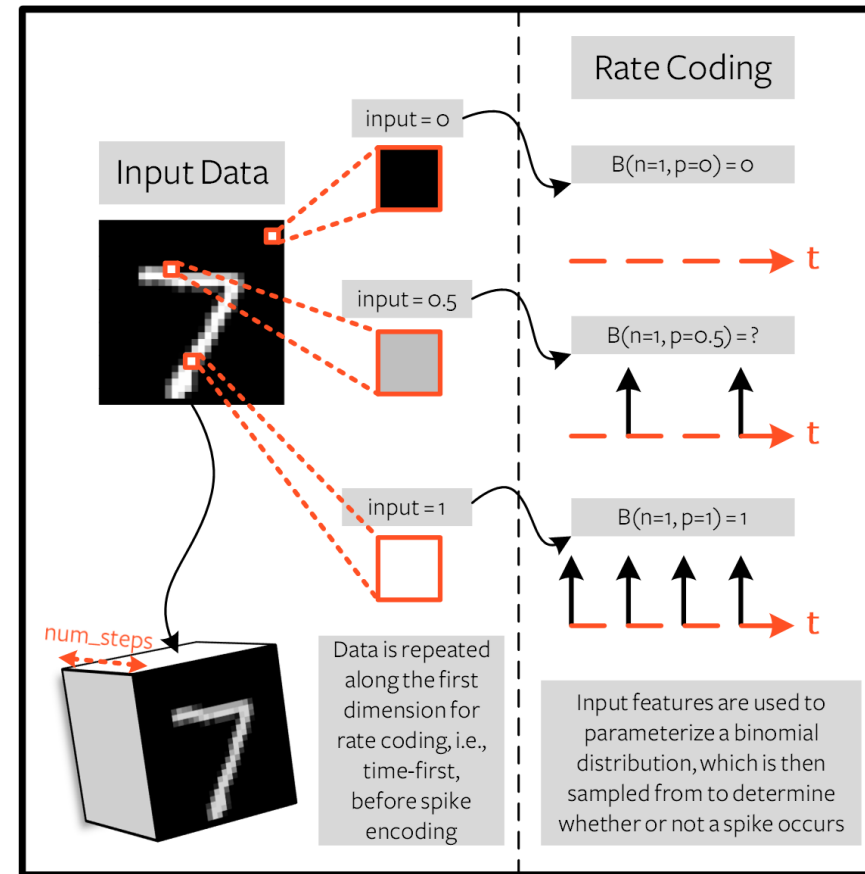
- Load and transform datasets with PyTorch.
- Create a PyTorch Dataloader.
- Implement convolutional SNN classifier with LIF neurons using [snnTorch](#), using the [nn.Model](#) class from Pytorch.
- Implement your own pipeline to train and evaluate the classifier.

Spike input encoding methods

- Input data do **not necessarily** have to be encoded into spikes.
- Continuous values can be treated as a **direct current** input and passed to the network at each timestep.
- 3 main spike encoding methods:
 - **Rate coding** → converts input intensity into a **firing rate** or **spike count**.
 - **Latency (or temporal) coding** → converts input intensity to a **spike time**.
 - **Delta modulation** → converts **temporal changes** of input intensity into spikes.

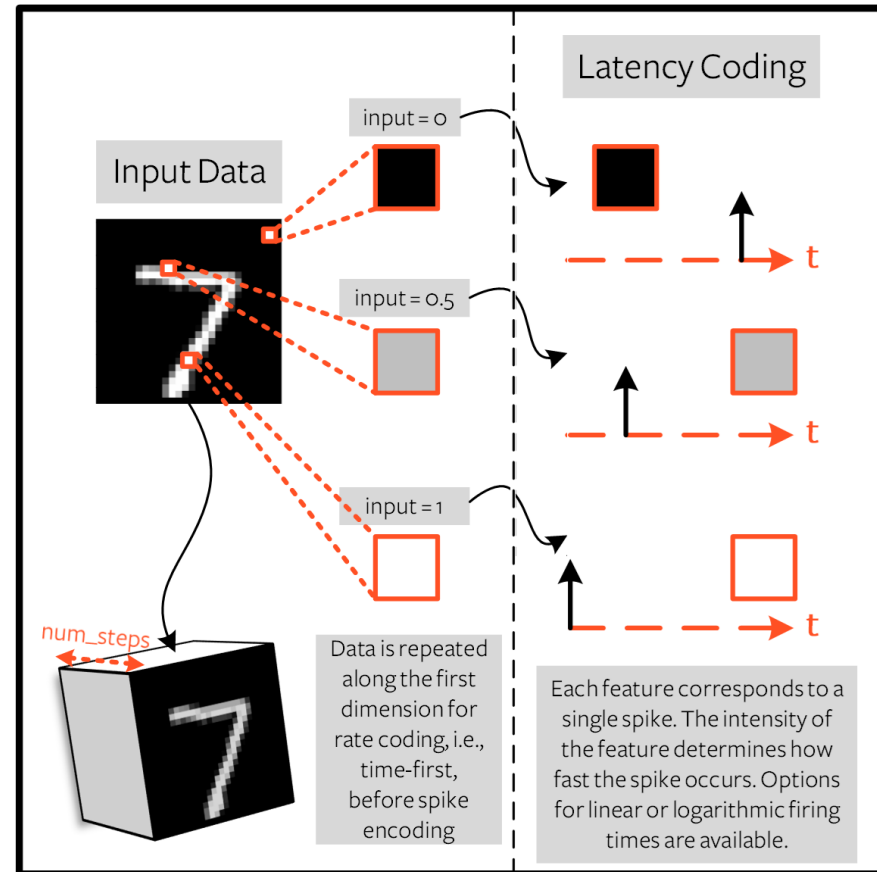
Rate coding (MNIST example)

- Each normalized input feature X_{ij} is used as the **probability a spike occurs at any given time step**.
- Bernoulli trial: $R_{ij} \sim B(n, p)$
(in our case: $n=1, p = X_{ij}$)
- The probability a spike occurs is:
$$P(R_{ij} = 1) = X_{ij} = 1 - P(R_{ij} = 0)$$
- For MNIST image:
 - a white pixel corresponds to a 100% probability of spiking;
 - a black pixel will never generate a spike.



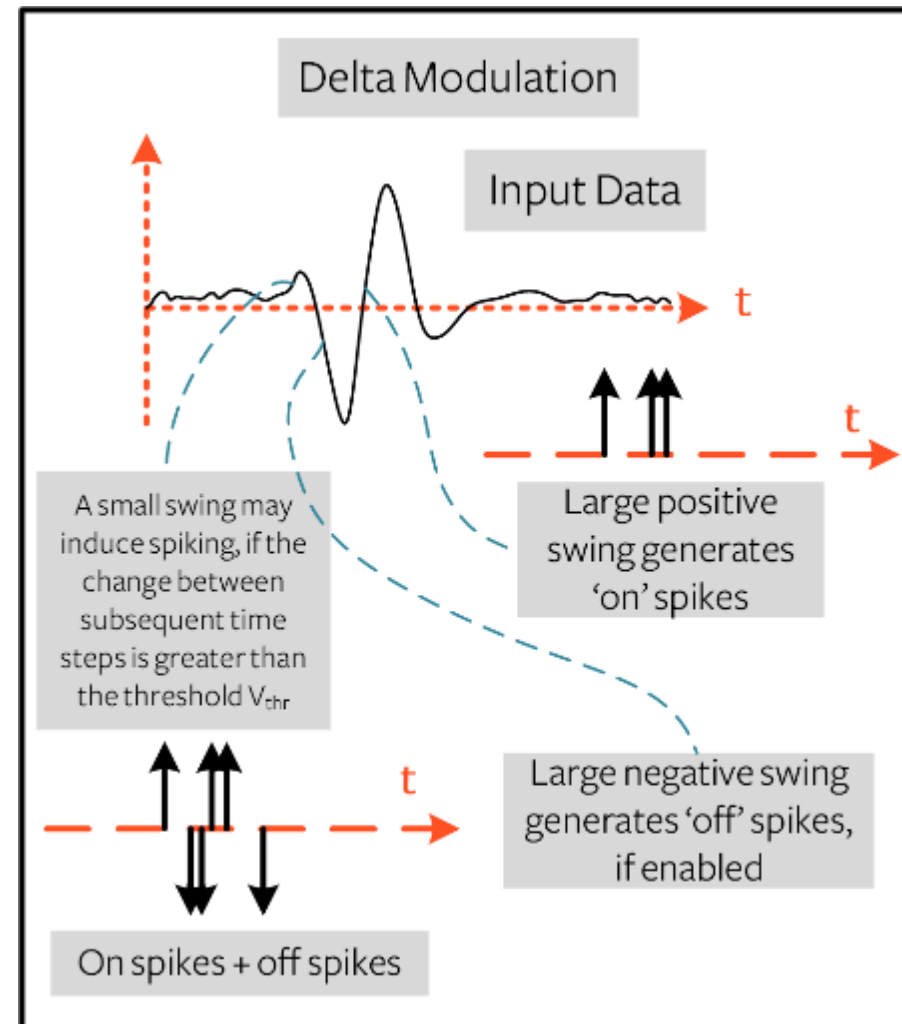
Latency coding (MNIST example)

- A single spike carries much more meaning than in rate codes which rely on firing frequency.
- Each input is allowed to fire **at most once** → input will be very sparse.
- Features closer to 1 will fire earlier and features closer to 0 will fire later.
- For MNIST image:
 - bright pixels will fire earlier;
 - dark pixels will fire later.



Delta modulation

- Input: time-varying data.
- Generates spikes by looking at the **difference** between two subsequent values across all time steps.
- If the difference is **greater** than a predefined (positive) *threshold*, a **positive spike** (+1) is generated.
- Accounts also for **negative spikes**, generated when the difference is **smaller** than *- threshold*.



Output decoding

- Each LIF neuron returns **two values**: the **output spikes** and the **membrane potential** (continuous values).
- Depending on the task, you can choose to use either the spikes or the membrane potential to train the network.
- The membrane potential can be used directly, for example with an MSE loss, to perform **regression tasks**.
- If you use **output spikes**, you need a method to **decode** them into meaningful predictions.
- In **classification tasks**, the spikes from the last layer are typically **accumulated over time**, producing a tensor of shape **(T, B, C)**, where **T** is the number of timesteps, **B** is the batch size, and **C** is the number of classes.

Output decoding

- **Rate coding** → chooses the output neuron with the highest **firing rate**, or **spike count**, as the predicted class.
- **Latency (or temporal) coding** → chooses the output neuron that **fires first** as the predicted class.
- **Population coding** → relies on **multiple neurons per class** (typically used in conjunction with rate coding).
- **Note**: the decoding method **does not need** to match the encoding method.

The architecture

- SNN with **3 layers**: one input layer, an hidden layer and a final output layer.
- Each layer in a SNN is a **composition** of two different layers:
 - a Pytorch layer to integrate the input (in this Lab we will use `nn.Conv2d` and `nn.Linear`)
 - and an `snn.Leaky` layer, which implements a *first-order leaky integrate-and-fire* neuron model.

LAB 6: SNN FOR CLASSIFICATION

Instructor

Michele Rossi - michele.rossi@unipd.it

Lab. classes

Eleonora Cicciarella - eleonora.cicciarella@phd.unipd.it

Cesare Bidini - cesare.bidini@phd.unipd.it

